Consolidation: lessons 1 to 5, one call at a time

Technologies for Artificial Intelligence — self-study, and two hours in class

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable

Contents

What this is . . . 1
How to use an atom . . . 2
The files . . . 2
The atoms . . . 3
00 — the scikit-learn API . . . 3
01 — lesson 1 . . . 3
02 — lesson 2 . . . 3
03 — lesson 3 . . . 4
04 — lesson 4 . . . 5
05 — lesson 5 . . . 5
Two hours in class . . . 6
What students will get wrong, and why it is reasonable . . . 7

What this is

Lessons 1 to 5 ran every scikit-learn call on hundreds or thousands of rows: 569 tumours, 2,000 customers, 600 houses, 8,000 disk drives. At that size the result of a call is visible, and what the call *did* is not, and the lesson notebooks often take in one line a step that someone new to machine learning needs to see on its own. This folder takes those steps one at a time, on two to ten rows, small enough that every number can be checked with a pencil.

Each section of a notebook is an **atom**: one idea, one call. Many atoms recompute the worked examples of the lesson 2, 3 and 4 commentaries (`Docs/*_commentary.md`) with the library, so that the arithmetic done by hand and the call that does it sit side by side; the numbers are the same.

An atom is here for one of two reasons: it shows a step that the lesson notebooks take without stopping on it (what `fit` stores, why `transform` and not `fit_transform` on the test set, what `predict` does with a probability), or it computes a worked example of a lesson's commentary or slides with the library call that does it. An idea the lesson notebook already shows in full is not repeated here.

It is **not examinable**, and it adds no new material. It does not replace the lesson notebooks either: every atom names the slide and the notebook section where its idea is treated at full scale.

Reading time: about 40 minutes for each of `00`, `02`, `03`, `04` and `05`, and 15 for `01`, so about three and a half hours for the six, done properly.

How to use an atom

Every atom has the same shape:

1. **What it shows**, in a sentence or two, and *From:* the lesson slide (by title) and notebook section it comes from.
2. **The data**, tiny, printed.
3. **Your prediction, before you run the next cell**: one concrete question.
4. **The code**: a few lines, with the call being taught in plain sight.
5. **What to look at**: what the output means, in the units of the problem.
6. **Try changing**: one small change to make and run.

The third step is the one that matters. Write the answer down, even as a single number, *before* running. A prediction that turns out wrong marks the exact place where your picture of the library differs from what it does, and that is worth more than any number of predictions that came out right.

The files

File	What it covers	Atoms
`00_the_scikit_learn_api.ipynb`	`fit`, `transform`, `fit_transform`, `predict`, `predict_proba` and the threshold, `score`, the shape of `X`, `random_state`, `Pipeline`, `decision_function`, the thresholds behind a curve, the defaults `get_params()` shows	11
`01_lesson_1_atoms.ipynb`	splitting and `stratify`, the baseline, the confusion matrix, precision and recall	4
`02_lesson_2_atoms.ipynb`	looking, missing values, outliers, scaling, encoding, `ColumnTransformer`, the area under the curve, two leaks	15
`03_lesson_3_atoms.ipynb`	cost, the normal equation, gradient descent, Ridge and Lasso, what a coefficient means	15
`04_lesson_4_atoms.ipynb`	odds and log-odds, log loss, one gradient step, the confusion matrix in order, the two averages, AUC and average precision, a threshold from costs, class weights, softmax	11
`05_lesson_5_atoms.ipynb`	cross-validation round by round, the five round models, the spread and the report line, stratified, repeated and grouped folds, training against validation error, `GridSearchCV`, nested cross-validation one round at a time, a fresh test set, the threshold sweep, seeds	13

They run in the course container like every other notebook, each in a few seconds, with no network. Start with `00`: the others assume it.

The atoms

Italic titles are worked examples of the lesson's commentary: same data, same numbers.

00 — the scikit-learn API

#	Atom	Lesson source
1	Estimators learn in `fit`, and keep what they learned	L1 *Step 5: pipeline, not two steps*; L2 *What to do about a gap*
2	`transform` applies what `fit` learned; `fit_transform` does both	L2 *StandardScaler vs MinMaxScaler*
3	`predict` gives a label, `predict_proba` the probabilities behind it, and 0.5 sits between them	L1 *Behind every label, a probability*; *The trade-off, drawn*
4	`score` is a metric with a default	L1 *Accuracy: the share of right answers*; L3 *A cost needs something to be compared against*
5	Why arrays, and why `X` has two dimensions	L1 notebook 2; L3 *θ and X: all parameters, all houses*
6	`random_state`: the same randomness, every time	L1 *One split is one measurement*
7	`Pipeline`: one `fit`, and every step fitted on the right rows	L1 *Step 5*; L2 *ColumnTransformer + Pipeline*
8	The contract, in one table	lessons 1 to 3
9	`decision_function`: the score before the sigmoid	L4 *Log-odds of p = P(y = 1 | x): a line*; *Three classes: softmax (optional)*
10	`roc_curve` and `precision_recall_curve` return the thresholds too	L4 *The precision-recall curve*; *The ROC curve*
11	`get_params()`: the defaults you did not choose	L4 *And it is convex, which squared error is not*; L4 notebook 1, section 6

01 — lesson 1

#	Atom	Lesson source
1	`train_test_split`, and what `stratify` keeps	*Step 3: split, before anything else*
2	`DummyClassifier`: what a baseline learns, and the score to beat	*Step 4: baseline first*
3	The confusion matrix: where the right and wrong answers fall	*What accuracy hides*
4	Precision and recall, counted and then printed	*Precision and recall, counted*

02 — lesson 2

#	Atom	Lesson source
1	Load and look	*Look before you touch anything*

#	Atom	Lesson source
2	Finding the missing values	*Where the gaps are*
3	Dropping rows and columns	*What to do about a gap*
4	Mean imputation: *four ages out of ten*	*What mean imputation actually costs*
5	The missing indicator: keeping the fact of the gap	*What to do about a gap*; *Choosing among the four*
6	Outliers: *the ruler made of the outlier*, and *the same eight values*	*Rule 1*; *Rule 2: interquartile range (IQR)*
7	Two scalers: *five customers, one outlier*	*StandardScaler vs MinMaxScaler*
8	*One step size for two directions*	*A 100:1 variance ratio, and what it costs*
9	*Ten customers, three encodings*, and *the steps ordinal encoding assumes equal*	*One customer, three encodings*; *Ordinal: equal steps, unequal churn*
10	*Four rows, four columns, rank three*	*One one-hot column too many*; *4 columns, but only rank 3*
11	`ColumnTransformer`: a different preparation for each column, and what each piece learned	*ColumnTransformer + Pipeline*; *The code*
12	*The AUC by counting pairs*	*The model, on churn*
13	Leak 1: *the two nearest neighbours are both in the test set*	*Leak 1: impute before splitting*
14	Leak 2: *a zip code with no signal, and an AUC of 0.875*	*Leak 2: encode before splitting*; *How much of its own label a row sees*
15	The fix: *the same eight customers, cross-fitted*, and `TargetEncoder`	*The fix*

03 — lesson 3

#	Atom	Lesson source
1	*The best single price for three houses*	*Why squared, not absolute*
2	The cost of one guess	*Trying $\hat{y}$ = 2,400 · area + 45,000*
3	*The same model, two test sets*	*A cost needs something to be compared against*
4	The design matrix, the label vector, and the normal equation by hand	*θ and X*; *The same equation, by hand*
5	*One slope, no intercept*: the cost is a bowl	*The cost is a bowl*
6	*Two standardised features, rising correlation*	*How stretched is the valley?*
7	*Three houses voting on the price of a square metre*	*The gradients: one for w, one for b*
8	Gradient descent, one iteration at a time	*One iteration, then the next*; *One step on two houses*
9	*A bowl of curvature 2*, and four learning rates	*Three learning rates*
10	*A parabola by the normal equation*	*"Linear" means linear in the coefficients*
11	*One coefficient, four penalties*	*Ridge has an exact solution too*
12	*The same column twice*: *one effect, two columns*	*The case Ridge was invented for*
13	*Three coefficients, both penalties*	*Ridge shrinks, Lasso selects*
14	*The same column, two coefficients*	*What a coefficient actually says*

#	Atom	Lesson source
15	When can you trust a coefficient?	*When can you trust a coefficient?*

04 — lesson 4

#	Atom	Lesson source
1	*Five probabilities, their odds and their log-odds*: `logit` and `expit`	*Why the log-odds*
2	*One coefficient, three drives*: odds, not probability	*What the coefficients mean*
3	*Four drives, one confident mistake*: `log_loss`	*Surprise: minus the log of a probability*; *Cross-entropy, also called log loss*
4	*One step of gradient descent on three drives*	*Lesson 3's gradient*
5	*Twenty drives, four of which fail*: `confusion_matrix(...).ravel()` in its order	*Four numbers instead of one*; *Our model, in those terms*
6	`classification_report`: the two averages, by hand	*Read the minority class, not the average*
7	*The AUC of seven drives*: ties count half, and only the order matters	*What the area means*; *What AUC does not tell you*
8	*Average precision of a seven-drive ranking*: a step sum, not a trapezoid	*The rule: report both*
9	*Invented costs: a false alarm €50, a missed failure €200*: a threshold applied to the probabilities	*Where the two costs cross*; *Where they cross, in one line*
10	*A weight ratio of 4*: `compute_class_weight` and the threshold it stands for	*What class weights do*; *Class weights: the same lever, pulled earlier*
11	*Softmax on three scores*, and `predict_proba` for three classes	*Three classes: softmax (optional)*

05 — lesson 5

Lesson 5 is about measuring, so most of its atoms run on the lesson's own data, the 800 drives and the 25 days, where the slides' numbers come from; atoms 1 and 5 use ten and twenty rows. Atoms 9 to 11 repeat notebook 3's search and take about a minute.

#	Atom	Lesson source
1	Cross-validation on ten days: who trains, who is held out; `neg_mean_squared_error`	*k-fold cross-validation*
2	Five round AUCs and their mean, 0.9514, by hand and with `cross_val_score`	*What it actually estimates*
3	Five rounds, five different models, and a sixth you ship	*Five rounds, five different models*
4	The spread: s by hand, `std()` against `std(ddof=1)`, and the report line	*Report the spread, but not as a confidence interval*; *What to put in a report*

#	Atom	Lesson source
5	`KFold` against `StratifiedKFold`, on twenty rows and on the 800 drives at seed 3	*Stratify when a class is rare*
6	`RepeatedStratifiedKFold`: the seed moves the score less	*Change only the seed, 40 times*; *How many folds?* (optional)
7	Ten readings per drive: `GroupKFold`, 1.000 against 0.620	*When the rows are not independent*; *Split by the thing you must generalise to*
8	Training against validation error with `cross_validate(..., return_train_score=True)`	*Without the truth: two errors you can still measure*
9	`GridSearchCV`: 25 candidates, `best_score_` before the refit	*GridSearchCV: try all, keep the best*; *The best of 25 scores is inflated*
10	Nested cross-validation, one outer round at a time	*Cross-validate the whole search*; *Nested cross-validation: it measures what choosing costs*
11	A fresh test set is centred on the truth	*Why it does not return to 0.5*
12	The threshold: a sweep of 200 candidates on the validation drives	*The debt, paid*
13	Seeds: the same seed, the same number, and still a spread	*A fixed seed buys repeatability, not stability*; *What to fix, and what to write down* (optional)

Two hours in class

A selection of thirteen atoms, run together from the projector. For each one: read the question aloud, give the room a minute to commit to an answer (a show of hands works for most), run the cell, then ask who was surprised and why. The surprise is the point; the explanation follows from it.

Minutes	Notebook · atom	What it settles
5	—	how an atom works: predict, then run
8	00 · 1	what `fit` learns and where it keeps it
10	00 · 2	`transform` against `fit_transform` on the test set
10	00 · 3	`predict` is a probability compared with 0.5, and moving the 0.5
7	00 · 7	a pipeline fits each step on the training rows
8	01 · 1	what `stratify` keeps, on eight patients
12	01 · 3 and 4	the confusion matrix, then precision and recall from it
10	—	break
10	02 · 9	three encodings of ten customers
7	02 · 12	the area under the curve as a count of pairs
12	02 · 14 and 15	the target-encoding leak, and the fix
8	03 · 4	the normal equation on three houses
10	03 · 8	gradient descent, one iteration at a time
3	—	close: which prediction surprised you most?
120		

Everything not in the selection is left for self-study, and the guide's index above is where to find it.

What students will get wrong, and why it is reasonable

- **"`fit_transform` on the test set is the same thing, just shorter."** The name invites it: if `fit_transform` is `fit` plus `transform`, using it everywhere looks like tidiness. On the training set it is; on the test set it learns from the rows being measured. Atom $00 \cdot 2$ shows a test customer at 2.24 standard deviations above average becoming exactly average.
- **"`predict` is the model's answer."** It is the model's probability compared with 0.5, a threshold nobody chose for the problem at hand, and it is natural to take a default for a decision. Atom $00 \cdot 3$ shows a malignant tumour at probability 0.21 that the default never flags, and what a threshold of 0.2 costs to catch it.
- **"A higher R^2 means a better model."** Only on the same test set. Atom $03 \cdot 3$ shows the model's error falling by more than two thirds while R^2 falls from 0.971 to 0.625, because the second test set has less spread to explain.
- **"`TargetEncoder` is target encoding, so it leaks."** It is the one class in scikit-learn where `fit_transform` is not `fit` followed by `transform`: it cross-fits, and atom $02 \cdot 15$ shows the two giving different columns. The leak comes back only if `transform` is called on the training rows by hand.
- **"`tn, fp, fn, tp` or `tp, fp, fn, tn`, it is the same four numbers."** The four names look interchangeable, and the matrix is small. But `.ravel()` reads the matrix row by row, truth in rows, so the order is fixed: TN, FP, FN, TP. Atom $04 \cdot 5$ shows that unpacking in the wrong order raises no error and swaps every metric.
- **"`LogisticRegression()` fits the plain log loss."** Nothing in the call says otherwise, and lesson 4 derived the plain log loss, so the reading is natural. But the default adds an L2 penalty with `C=1.0`. Atom $00 \cdot 11$ shows it shrinking a coefficient from 0.971 to 0.685 on eight rows, a gap that almost vanishes on 6,000, and `get_params()` showing every default before it matters.
- **"A high AUC means the probabilities can be trusted."** The AUC is presented as the best single summary of a classifier, so it is natural to read it as a guarantee. Atom $04 \cdot 7$ divides every score by ten: the AUC stays 0.792 while every decision at 0.5 changes, because the AUC sees only the order.
- **"`cross_val_score` returns the error."** For a regression it returns the error with a minus sign: its scorers are all "higher is better", so `neg_mean_squared_error` makes a larger error a smaller number. The name says so, and it is still easy to read past. Atom $05 \cdot 1$ shows five MSEs from 274.6 to 1,569.0 kWh^2 coming back as -274.6 to $-1{,}569.0$.
- **"`best_score_` is the AUC of the model `GridSearchCV` hands back."** The refit is the last thing the search does, so it is natural to think the score belongs to it. But the score is the winner's mean over the folds, computed before the refit, and the refitted model is never scored. Atom $05 \cdot 9$ shows `best_score_` equal to the highest of the 25 means, 0.7999, on data with no signal at all.